Le réseau de transport d'électricité

UNIVERSITÉ PARIS SUD

inventeurs du monde numérique

Graph Neural Solver for Power Systems

Balthazar Donon, Benjamin Donnot, Antoine Marot, Isabelle Guyon

Authors

Balthazar Donon[1,2]

Benjamin Donnot[1,2]

Antoine Marot[2]

Isabelle Guyon[1]

1 : TAU group of Laboratoire de Recherche en Informatique, ***INRIA / Université Paris-Saclay****,* Paris, France

2 : *R&D department,* ***Réseau de Transport d'Électricité (RTE)****,* Paris, France

Blackouts!

Blackouts are not a thing of the past

Background & Motivations

A changing environment

- Renewable energies
- Consumer's behavior
- Open market

➔ Need for reliability!

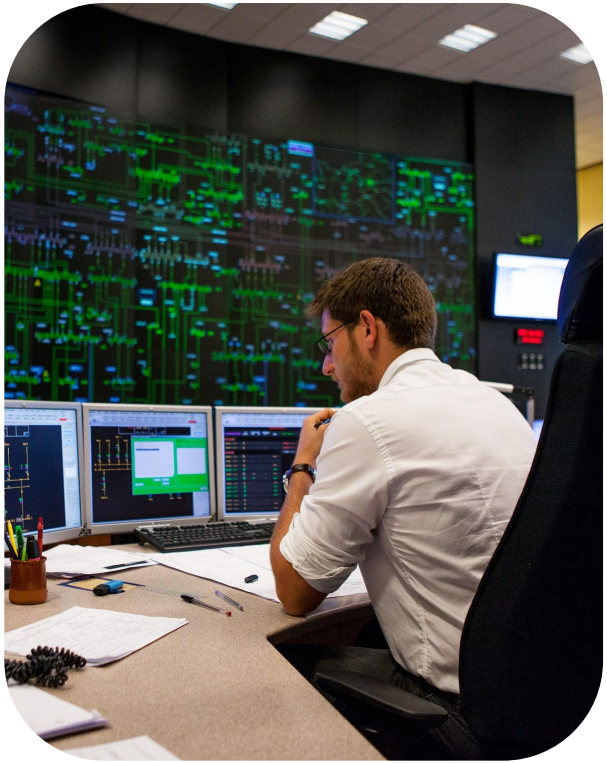
A changing environment

System security :

➔ Expensive “load flow” simulations

OR

➔ Develop approximations (proxies)



Load flow

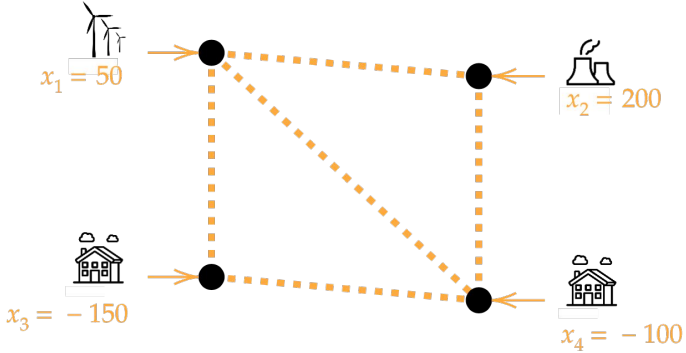
X

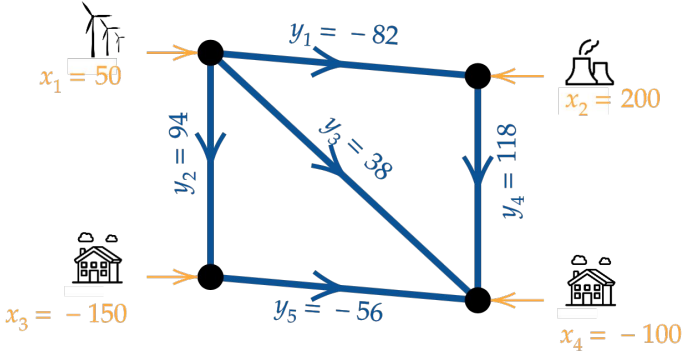
injections*
&
topology**

Physics

Y

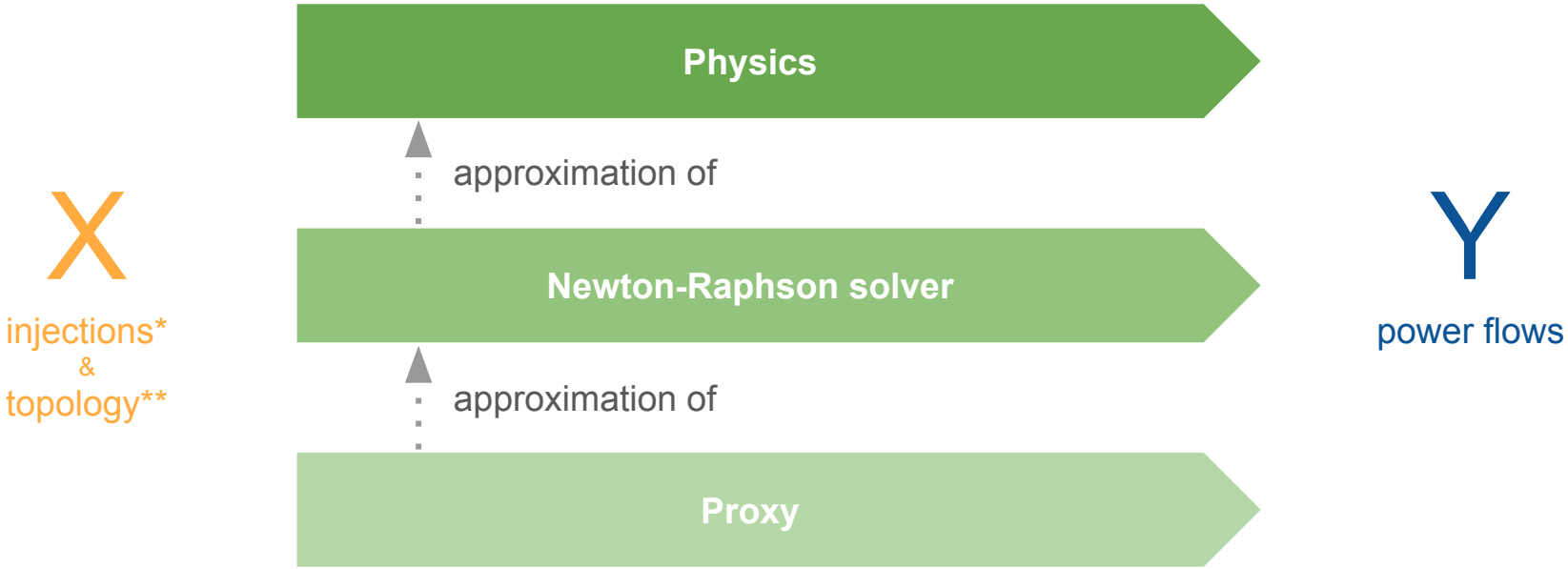
power flows





*productions & consumptions
**interconnections

Load flow



*injections = productions & consumptions
**topology = interconnections between the injections

Generalization issue

Robustness w.r.t. graph perturbation?

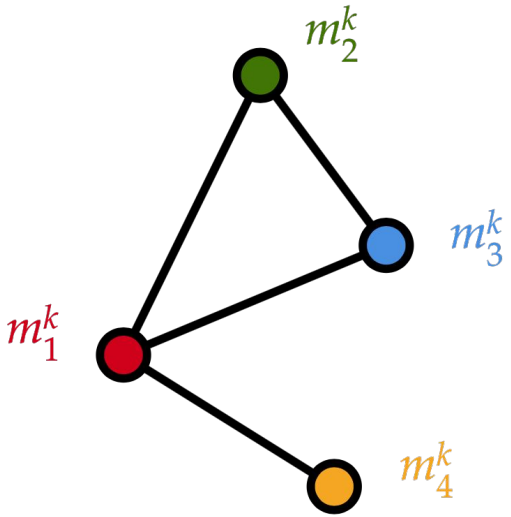
Fully Connected	LEAP net[1]	Graph Neural Solver
NO	Yes but...	YES

[1] **LEAP nets for power grid perturbations**, B. Donnot et al, ESANN Proceedings 2019

Graph Neural Networks

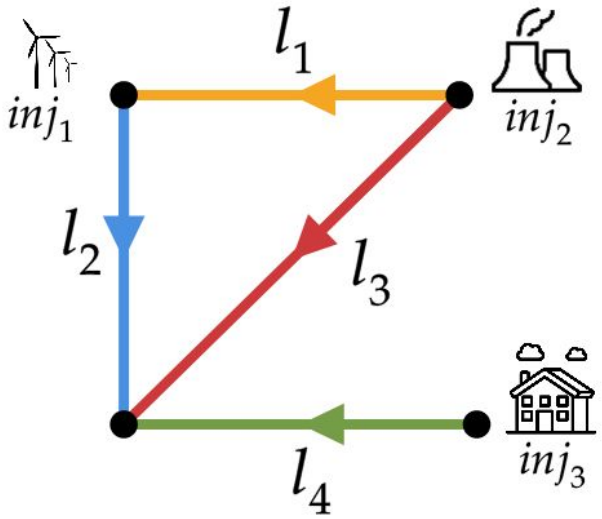
- Message propagation through edges
- Exploit graph structure!

$$m_i^{k+1} = m_i^k + L(\sum_{j \in Neighbors(i)} m_j^k)$$

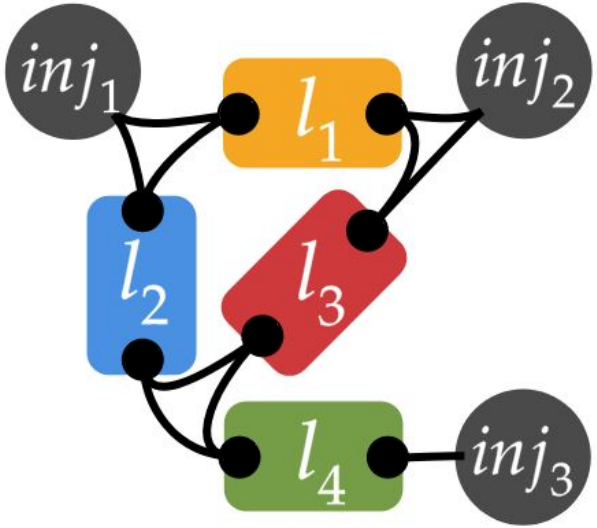


Proposed Methodology

Graph of Power Lines

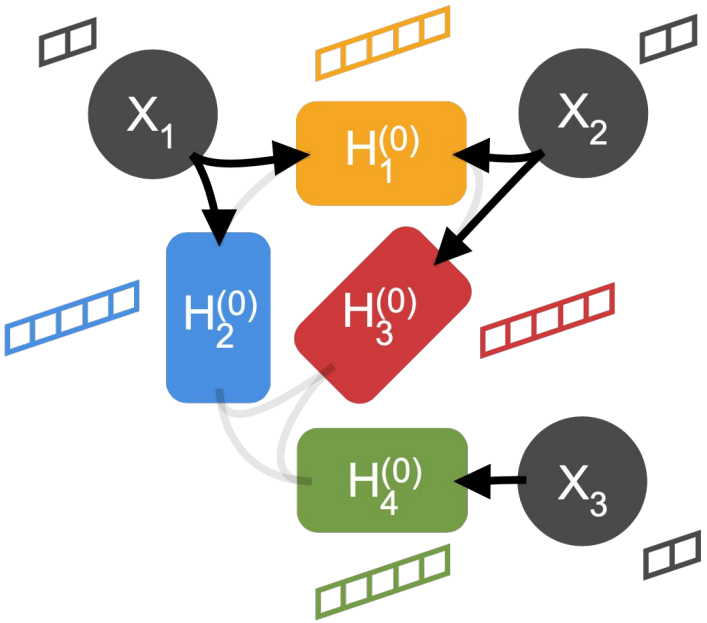


Graph of electrical nodes



Graph of transmission lines

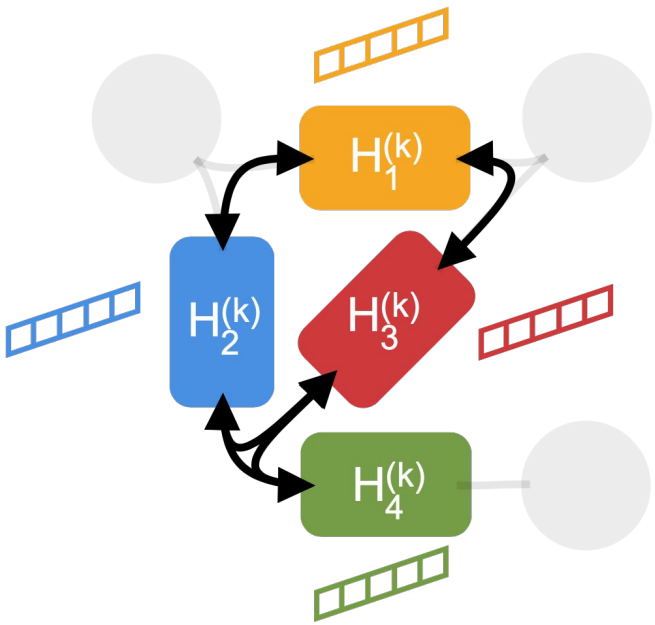
Embedding



$$H^{(0)} = \mathbf{A}_{inj} \circ \mathbf{E}(X)$$

$\mathbf{A}_{inj}$: Sends messages from injection to power line

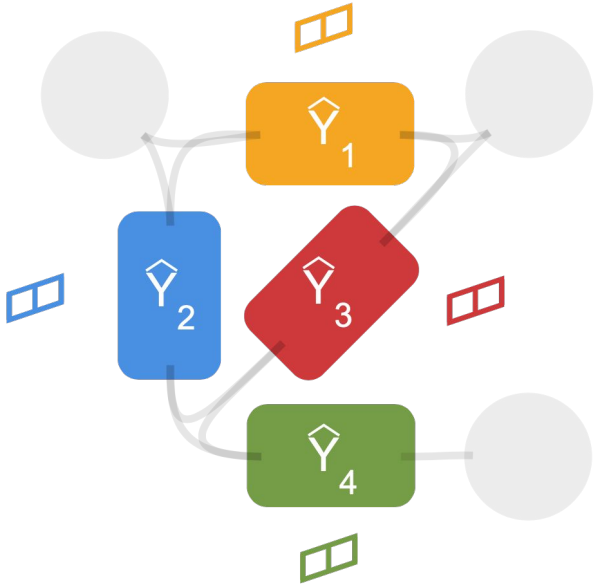
Propagation



$$H^{(k+1)} = H^{(k)} + \mathbf{L}^{(k)}(\mathbf{A}(H^{(k)}))$$

$\mathbf{A}$: Power line adjacency matrix

Decoding



$$\hat{Y} = \mathbf{D}(H^{(K)})$$

Overall Architecture

inputs : X, A_{inj}, A

$$H^{(0)} = \mathbf{A}_{inj} \circ \mathbf{E}(X)$$

Embedding

for k **from** 0 **to** $K-1$ **:**

$$H^{(k+1)} = H^{(k)} + \mathbf{L}^{(k)}(\mathbf{A}(H^{(k)}))$$

Propagation

$$\hat{Y} = \mathbf{D}(H^{(K)})$$

Decoding

output : $\hat{Y}$

Overall Architecture

inputs : X, A_{inj}, A

$$H^{(0)} = \mathbf{A}_{inj} \circ \boxed{\mathbf{E}}(X)$$

Embedding

for k **from** 0 **to** $K-1$ **:**

$$H^{(k+1)} = H^{(k)} + \boxed{\mathbf{L}^{(k)}}(\mathbf{A}(H^{(k)}))$$

Propagation

$$\hat{Y} = \boxed{\mathbf{D}}(H^{(K)})$$

Decoding

output : $\hat{Y}$

Learnable neural network blocks

Design Characteristics

- Requires a full topological description
- Message dimension is a hyperparameter
- Only local exchange
- Neural blocks independent from power grid

Experiments

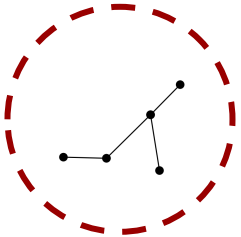
Experimental setup

- Baselines :
  - Fully Connected neural network
  - DC approximation

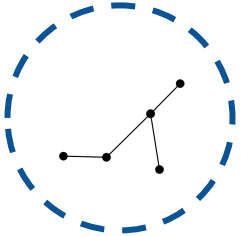
- Metrics :
  - l2 - loss (objective)
  - MAPE90A (% of error)

Experiment A

Train set : Fixed 30-nodes grid.



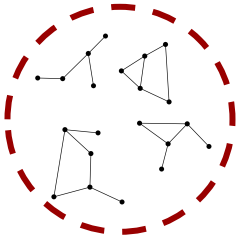
Test set : Same grid.



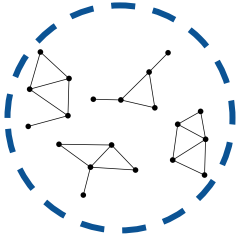
	Fully Connected	**proposed GNS**	DC-approx.
Loss	0.049 +/- 0.001	**0.03 +/- 0.01**	0.68
MAPE90A (% of error)	0.53 +/- 0.03	**0.47 +/- 0.05**	3.2

Experiment B

Train set : Random 30-nodes grids.



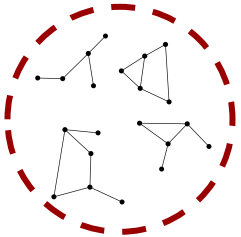
Test set : Random 30-nodes grids.



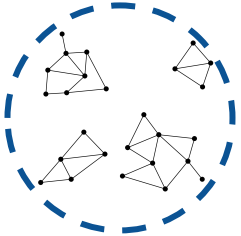
	Fully Connected	**proposed GNS**	DC-approx.
Loss	NA	**0.07 +/- 0.01**	0.68
MAPE90A (% of error)	NA	**0.73 +/- 0.05**	3.2

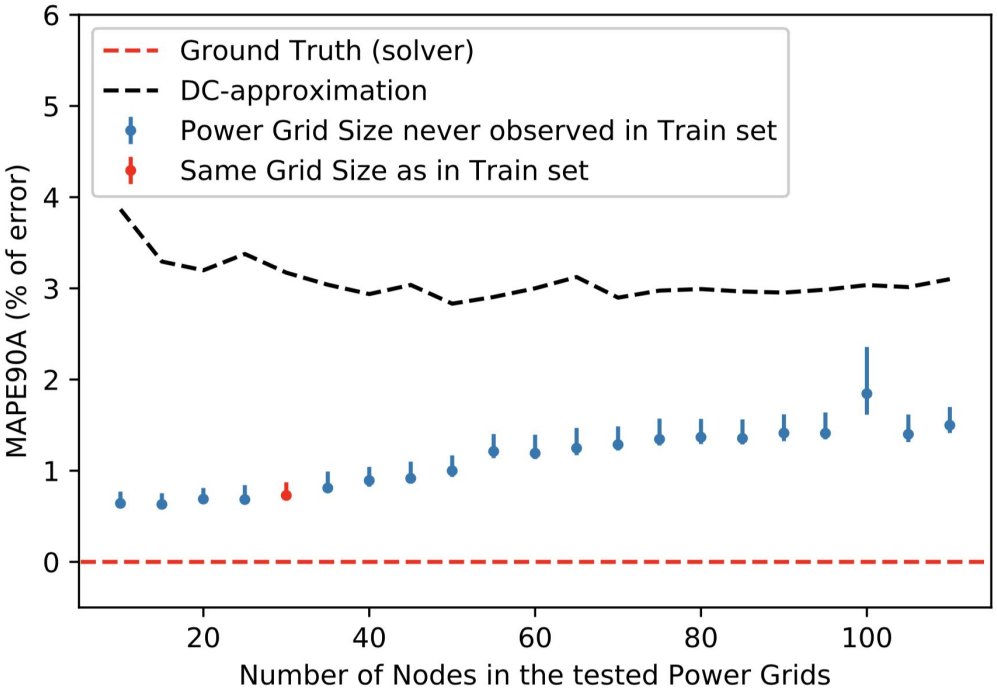
Experiment C

Train set : Random 30-nodes grids.



Test set : Random grids of various sizes.





Discussion & Conclusions

Conclusion

- Learn underlying physics only from observations
- **Invariances & symmetries** directly encoded
- Zero-shot learning
- Beats DC-approximation

Discussion

- Computational time
- Limiting assumption
- Transient system
- Finite elements systems?

Future work ...

- Unsupervised learning?
- Faster sparse implementation?
- Interesting for control?

Permutation Invariance

⇒ Graph Neural Network

Read our paper!

https://tinyurl.com/graph-neural-solver